

PROJECT EXPLANATION GUIDE

DevTask

A complete walkthrough of the design, architecture, database, backend code, frontend code, and interview-ready explanations for the DevTask project.

Generated on 9 May 2026

1. Project Summary

DevTask is a full-stack Jira-like task management application. It helps teams create projects, manage tasks, plan sprints, collaborate through comments and notifications, and review progress through reports.

One-line explanation: DevTask is a React, Express, PostgreSQL, and Prisma based project management app with authentication, RBAC, Kanban boards, sprint planning, comments, notifications, and analytics.

Layer	Technology	Responsibility
Frontend	React + Vite	UI, routing, state, API calls, protected pages, dashboards, board, backlog, reports.
Backend	Node.js + Express	REST API, auth, authorization, validation, business logic, sockets, security middleware.
Database	PostgreSQL + Prisma	Persistent storage, relations, indexes, transactions, query abstraction.
Realtime	Socket.io	Foundation for live task and notification updates.

2. High-Level Architecture

The project follows a clear separation between frontend and backend. The frontend calls backend APIs through Axios. The backend validates and authorizes requests before calling service functions. Services use Prisma to read and write PostgreSQL data.

```
React Page or Component
  → Axios API Client
  → Express Route
  → Auth Middleware
  → RBAC / Resource Access Middleware
  → Zod Validation
  → Controller
  → Service
  → Prisma ORM
  → PostgreSQL
```

This structure is useful because UI logic, request handling, security, and business logic stay in different places.

3. Database Design: backend/prisma/schema.prisma

This file defines the complete data model. Prisma uses it to generate a typed database client and manage migrations.

Model	Purpose
User	Stores user name, email, hashed password, comments, assigned tasks, activities, projects, and notifications.
Project	Represents a team project. It owns tasks, sprints, labels, and members.
ProjectMember	Join table between users and projects. Stores role such as ADMIN or MEMBER .
Task	Main issue model. Stores title, description, type, status, priority, assignee, sprint, labels, hierarchy, rank, comments, and activities.
Sprint	Groups tasks into planned cycles with status PLANNED , ACTIVE , or COMPLETED .
Comment	Stores task discussions with author and task relation.
Label and TaskLabel	Many-to-many label system scoped to each project.
TaskActivity	Audit trail of task changes such as creation, status changes, and assignment.
Notification	Stores user notifications with title, message, type, read state, and link.

Important design points: task numbers are unique per project using `@@unique([projectId, taskNumber])` ; indexes are added for common filters like project, assignee, status, sprint, and priority-related workflows; cascading deletes remove dependent records when a project or task is deleted.

4. Backend Entry Point: backend/server.js

This is the main server file. It loads environment variables, creates the Express app, registers middleware, mounts API routes, starts the HTTP server, initializes Socket.io, and handles graceful shutdown.

- `helmet()` adds security headers.
- `express.json()` parses JSON request bodies.
- `cookieParser()` reads cookies, including the auth token cookie.
- `cors()` allows frontend requests from the configured frontend URL.
- `express-rate-limit` protects general API routes and login/register endpoints.
- Routes are mounted under `/api/auth` , `/api/projects` , `/api/tasks` , `/api/sprints` , and `/api/notifications` .
- The global error handler is registered last.
- Shutdown logic closes the HTTP server and disconnects Prisma.

How to explain it: `server.js` is the composition root of the backend. It wires security, parsing, CORS, rate limiting, routing, error handling, realtime setup, and process shutdown.

5. Authentication: `authController.js` and `authMiddleware.js`

`backend/src/controllers/authController.js`

The auth controller handles register, login, logout, current user, and password change operations. It delegates user creation and login checks to the auth service. On login, it returns the user and token, and also sets an HTTP-only cookie.

- `register` : creates a new user and returns the new user id.
- `login` : verifies credentials, creates JWT, sets cookie, returns user and token.
- `getMe` : fetches current user profile from `req.user`.
- `changePassword` : checks current password with bcrypt, hashes the new password, and updates the user.
- `logout` : clears the token cookie.

`backend/src/middlewares/authMiddleware.js`

The `protect` middleware protects private routes. It accepts tokens from either the `Authorization` header or the cookie. It verifies the JWT using `JWT_SECRET` and attaches the user id to `req.user`.

Interview answer: Authentication is JWT-based. The backend verifies the token on protected routes and attaches the authenticated user id to the request, so later middleware and controllers know who is performing the action.

6. Authorization: `rbacMiddleware.js` and `resourceAccessMiddleware.js`

`backend/src/middlewares/rbacMiddleware.js`

This file implements role-based access control. `resolveProjectRole` finds the user's membership for the project and stores the role in `req.userRole`. `authorize` checks whether that role is allowed for the route.

`backend/src/middlewares/resourceAccessMiddleware.js`

This file protects nested resources such as tasks and sprints. For example, updating a task only gives the route a `taskId`, so the middleware first finds that task, discovers its project, checks whether the user belongs to that project, then attaches the role to the request.

Why this matters: users cannot update or delete tasks from projects they do not belong to. This turns the app from basic CRUD into a team-safe product.

7. Validation: `backend/src/validations/schemas.js`

This file defines Zod schemas for incoming API data. It validates auth, project, task, sprint, member, and comment payloads before business logic runs.

- Auth schemas validate email format, password length, and required fields.

- Project schemas enforce uppercase alphanumeric project keys.
- Task schemas enforce valid UUIDs, task status, priority, issue type, labels, dates, and title.
- Sprint schemas validate project id, name, dates, and optional goal.
- `.strict()` rejects unexpected fields, which reduces accidental or malicious input.

8. Task API: `taskRoutes`, `taskController.js`, and `taskService.js`

The task feature is the best example of the backend architecture.

```
POST /api/tasks
  → protect
  → resolveProjectRole
  → authorize("ADMIN", "MEMBER")
  → validateSafe(createTaskSchema)
  → taskController.createTask
  → taskService.createTask
  → Prisma transaction
```

`backend/src/controllers/taskController.js`

The controller extracts request data and calls the service. It also sets pagination headers such as `X-Total-Count` and `X-Total-Pages` when returning task lists.

`backend/src/services/taskService.js`

The service contains the real task business logic:

- Checks that an assignee is a project member.
- Checks that sprint, epic, and parent task belong to the same project.
- Creates per-project task numbers inside a Prisma transaction.
- Creates task labels with `connectOrCreate`.
- Creates task activity records.
- Creates notifications when a task is assigned or updated.
- Supports filtering by sprint, backlog, type, search, assignee, and priority.
- Updates only changed fields and records meaningful activity history.

Strong explanation: Task creation uses a Prisma transaction because task number generation, task insert, label creation, activity logging, and notifications are part of one logical operation.

9. Project and Sprint Services

`backend/src/services/projectService.js`

This service manages project creation, lookup, update, deletion, members, and member notifications. When a project is created, the creator is automatically added as `ADMIN`.

- `createProject` : creates project and creator membership together.
- `getUserProjects` : returns only projects where the user is a member.
- `getProjectById` : checks project membership before returning project data.
- `addMemberByEmail` : finds user by email, prevents duplicates, creates membership, sends notification.
- `removeMember` : removes a member but prevents an admin from removing themselves.

backend/src/services/sprintService.js

This service manages sprint planning. A sprint belongs to one project and can contain many tasks.

- Creates planned sprints.
- Fetches sprints with their tasks and task metadata.
- Updates sprint status and dates.
- Moves tasks back to backlog before deleting a sprint.
- Checks task and sprint project match before adding a task to a sprint.

10. Frontend Routing: frontend/src/App.jsx

`App.jsx` defines all application routes using React Router. Public routes include home, login, and register. Private app routes are wrapped in `ProtectedRoute`.

Route	Page	Purpose
<code>/</code>	Home	Public landing/home page.
<code>/register</code>	Register	User signup.
<code>/login</code>	Login	User login.
<code>/dashboard</code>	Dashboard	Protected project dashboard.
<code>/project/:id</code>	Board	Protected Kanban board.
<code>/project/:id/backlog</code>	Backlog	Protected backlog and sprint planning page.
<code>/reports</code>	Reports	Protected analytics page.
<code>/profile</code>	ProfilePage	Protected account settings.

The app also wraps routes in toast and confirmation providers, making feedback and confirmation dialogs globally available.

11. API Client and Auth State

frontend/src/api/axios.js

This file creates the shared Axios client. It sets the backend base URL and enables credentials for cookies. A request interceptor reads the JWT token from local storage and attaches it as a Bearer token.

frontend/src/context/AuthContext.jsx

This context stores the current logged-in user. It initializes from local storage, provides `login` and `logout`, and clears local auth state when the user logs out.

How to explain it: the frontend keeps auth state globally through React Context, while Axios automatically attaches auth credentials to API requests.

12. Main App Layout: DashboardLayout.jsx

This component provides the authenticated app shell: sidebar navigation, mobile menu, notification entry point, profile block, logout button, and main content area.

- Reads current user through `/auth/me`.

- Uses `NotificationContext` to display unread notification count.
 - Uses React Router links for dashboard, notifications, reports, and profile.
 - Supports collapsible desktop sidebar.
 - Supports mobile sidebar overlay.
 - Calls logout and redirects to login.
-

13. Board Page: `frontend/src/pages/Board.jsx`

The board is the Kanban interface. It fetches the selected project, tasks, sprints, and members. It then renders columns for `TODO`, `IN_PROGRESS`, and `DONE`.

- Uses `useParams` to read project id from the URL.
- Fetches sprints and determines the active sprint.
- Fetches members for filters and assignment UI.
- Uses filters for search, assignee, priority, and issue type.
- Debounces search to avoid too many API calls.
- Shows skeleton loading state and an error state.
- Supports task creation, update, deletion, and detail modal.
- Implements drag-and-drop status updates.

Optimistic update: when a task is dragged to a new column, the UI updates immediately. If the API call fails, the task is moved back to its old status and a toast error is shown.

14. Backlog Page: `frontend/src/pages/Backlog.jsx`

The backlog page is for sprint planning. It separates unplanned tasks from sprint tasks. A task with no `sprintId` is treated as backlog work.

- Fetches project, sprints, backlog tasks, and project members together.
- Uses `sprintId=backlog` to request tasks without a sprint.
- Allows sprint creation and sprint status updates.
- Allows deleting a sprint and moving its tasks back to backlog.
- Allows moving tasks into and out of sprints.
- Keeps local UI state synchronized after task updates and deletions.

Product explanation: the backlog page supports agile planning by letting the team decide what should remain unplanned and what should be committed to a sprint.

15. Reports Page: `frontend/src/pages/Reports.jsx`

The reports page pulls project and task data, then calculates useful product metrics on the frontend.

- Fetches all accessible projects.

- Fetches tasks for each project.
- Computes total, open, completed, overdue, due soon, and completion percentage.
- Builds status, priority, project workload, and throughput datasets.
- Uses Recharts for bar charts, pie charts, and area charts.
- Supports filters by project and date range.
- Supports CSV export.

Scalability note: calculating reports on the frontend is fine for an MVP. For larger teams, move report aggregation to backend endpoints or materialized database views.

16. Complete Request Flow Example

Example: user drags a task from `TODO` to `DONE`.

1. `Board.jsx` captures the drop event.
2. The task status updates immediately in local React state.
3. Frontend sends `PATCH /api/tasks/:taskId` with the new status.
4. `protect` verifies the JWT.
5. `verifyTaskAccess` checks project membership for the task.
6. `authorize` checks that the user is `ADMIN` or `MEMBER`.
7. `validateSafe(updateTaskSchema)` validates the request body.
8. `taskController.updateTask` calls `taskService.updateTask`.
9. The service updates the task through Prisma.
10. The service creates an activity record such as status changed.
11. If needed, the service creates a notification.
12. The updated task returns to the frontend.
13. If the API fails, the frontend rolls back the optimistic UI change.

17. Interview-Ready Answers

What is the design pattern of the backend?

The backend uses a layered architecture: routes define endpoints, middleware handles auth and validation, controllers handle HTTP request and response, services contain business logic, and Prisma handles database access.

How do you prevent unauthorized access?

Every protected route uses JWT middleware. Project-level routes resolve the user's project role. Nested task and sprint routes first discover the parent project, then verify membership. Finally, role middleware checks whether the user's role is allowed.

Why did you use Prisma?

Prisma makes database queries safer and easier to maintain. It gives a clear schema, relation modeling, indexes, migrations, transactions, and a generated client for PostgreSQL access.

How does the Kanban board work?

The board fetches tasks for a project or active sprint and groups them by status. Drag-and-drop updates the task status through a PATCH API. The UI uses optimistic updates for speed and rolls back on failure.

How are sprints implemented?

Each sprint belongs to a project. Tasks can optionally reference a sprint. Tasks without a sprint are backlog tasks. Starting or completing a sprint changes the sprint status. Deleting a sprint moves tasks back to

backlog instead of deleting them.

What makes this more than a CRUD app?

It includes project membership, RBAC, resource-level access checks, sprint planning, Kanban workflow, comments, task activity history, notifications, filtering, reports, CSV export, security middleware, and deployment-ready frontend/backend separation.

18. Files Covered

- `backend/prisma/schema.prisma`
- `backend/server.js`
- `backend/src/middlewares/authMiddleware.js`
- `backend/src/middlewares/rbacMiddleware.js`
- `backend/src/middlewares/resourceAccessMiddleware.js`
- `backend/src/validations/schemas.js`
- `backend/src/controllers/authController.js`
- `backend/src/controllers/taskController.js`
- `backend/src/services/projectService.js`
- `backend/src/services/taskService.js`
- `backend/src/services/sprintService.js`
- `frontend/src/App.jsx`
- `frontend/src/api/axios.js`
- `frontend/src/context/AuthContext.jsx`
- `frontend/src/layouts/DashboardLayout.jsx`
- `frontend/src/pages/Board.jsx`
- `frontend/src/pages/Backlog.jsx`
- `frontend/src/pages/Reports.jsx`